DeepSentinel: Deepfake Face Detection with EfficientNet-B4, Grad-CAM Interpretability, and Cloud Deployment

Aditya Chaudhary
Purdue University
github.com/adichaudhary/deepsentinel • huggingface.co/spaces/adichaudhary/deepsentinel

October 2026

Abstract

DeepSentinel is an end-to-end system for detecting manipulated faces in images. An ImageNet-pretrained EfficientNet-B4 is fine-tuned as a binary classifier on face crops with identity-disjoint training, validation, and test splits, so reported results reflect performance on people the model has never seen. On a held-out, class-balanced test set of 4,000 images the model reaches **0.956 ROC AUC** and **88.8% accuracy**. Grad-CAM is used to inspect which facial regions drive each prediction, and a masking ablation rules out a suspected shortcut. The trained model is served through a containerized FastAPI service with OpenCV face detection, deployed on AWS ECS Fargate, and exposed through an interactive Gradio demo on Hugging Face Spaces.

1 Introduction

Face manipulation tools have made it easy to produce convincing synthetic faces, which creates risk for identity verification, media integrity, and fraud prevention. Many deepfake detectors report strong numbers that do not hold up in practice because the same identities appear in both training and test data. In that setup a model can learn to recognize people rather than manipulation artifacts.

DeepSentinel addresses this in three ways. It evaluates only on identities that never appear during training. It uses Grad-CAM to make predictions inspectable and tests whether a suspicious activation pattern reflects a learned shortcut. And it carries the model through to production as a REST API on AWS and a public demo, using the same preprocessing at inference time as during evaluation.

2 Data

The model is trained on the Deepfake Identity Isolated Dataset [1], released under CC BY 4.0. It contains about 143K pre-cropped 260×260 face images labeled real or fake, split into training (99,910), validation (21,611), and test (21,316) sets with no identity overlap across splits.

For this work each split is subsampled to an equal number of real and fake images: 10,000 per class for training and 2,000 per class for validation and test. Balanced sampling keeps accuracy, precision, and recall directly interpretable. Labels are remapped so that $y = 1$ denotes a manipulated face.

3 Method

3.1 Model

The classifier is EfficientNet-B4 [2] initialized from ImageNet weights via `timm` [4], with the classification head replaced by a single logit z. The predicted probability that a face is manipulated is $p = \sigma(z)$, and the model is trained with binary cross-entropy:

$$\mathscr{L} = -\big[y \log p + (1-y)\log(1-p)\big]. \tag{1}$$

3.2 Preprocessing and Augmentation

Each 260×260 crop is first center-cropped to 208×208 to remove image borders, then resized to 224×224 and normalized with ImageNet statistics. Training adds random resized crops (scale 0.8 to 1.0), horizontal flips, color jitter, Gaussian blur, and random erasing. Evaluation and deployment use only the deterministic center crop, resize, and normalization.

3.3 Training

The network is optimized with AdamW (learning rate 3×10^{-4}, weight decay 10^{-4}) under a one-cycle learning rate schedule for 5 epochs with batch size 64. Training uses FP16 mixed precision with gradient scaling. The checkpoint with the highest validation ROC AUC is kept as the final model. Full optimizer, scheduler, and scaler state is checkpointed every epoch, so interrupted runs resume automatically. Training ran on a single NVIDIA GPU on Purdue's Scholar cluster, and metrics, configuration, and figures were logged to Weights & Biases.

4 Results

Metric	Validation	Test
ROC AUC	0.954	**0.956**
Accuracy	87.4%	**88.8%**
Precision	89.5%	**90.2%**
Recall	84.8%	**87.0%**
F1	0.871	**0.886**

Table 1: Classification performance at a threshold of 0.5. Validation figures are from the selected checkpoint (epoch 5). The test set contains 4,000 balanced images from identities not seen during training.

Table 1 summarizes performance. Test results closely track validation results, which indicates the model generalizes to new identities rather than overfitting to the validation set used for checkpoint selection. Precision is somewhat higher than recall, so at the default threshold the model is slightly conservative about labeling a face as fake. Validation AUC rose steadily from 0.893 after the first epoch to 0.954 after the fifth.

5 Interpretability and Robustness

Grad-CAM [3] computes class activation maps from the final convolutional layer. Figure 1 shows that for many faces the model attends to the eyes, nose, and cheeks, which are regions where face-swap blending artifacts commonly appear.



Figure 1: Grad-CAM on held-out test images. Top row: inputs with ground-truth label and predicted probability of manipulation. Bottom row: class activation maps for the "fake" output.

The maps also show a recurring hot spot in the top-right corner, including on real faces. This could mean the model learned a shortcut such as a watermark or encoding artifact in that region. Two checks were run to test this:

1. **Border removal.** The model was retrained with a center crop that discards the outer border of every image. The corner activation persisted, even though the original corner pixels were no longer part of the input.
2. **Masking ablation.** The top-right quarter of every test image was zeroed out at inference time. Test ROC AUC moved from 0.9557 to 0.9554.

Because removing the region has no effect on performance, the corner activation is a boundary effect of the coarse 7×7 feature map that Grad-CAM upsamples, not information the classifier relies on.

6 Deployment

The trained model is packaged for production use and public demonstration. Figure 2 shows the overall architecture.

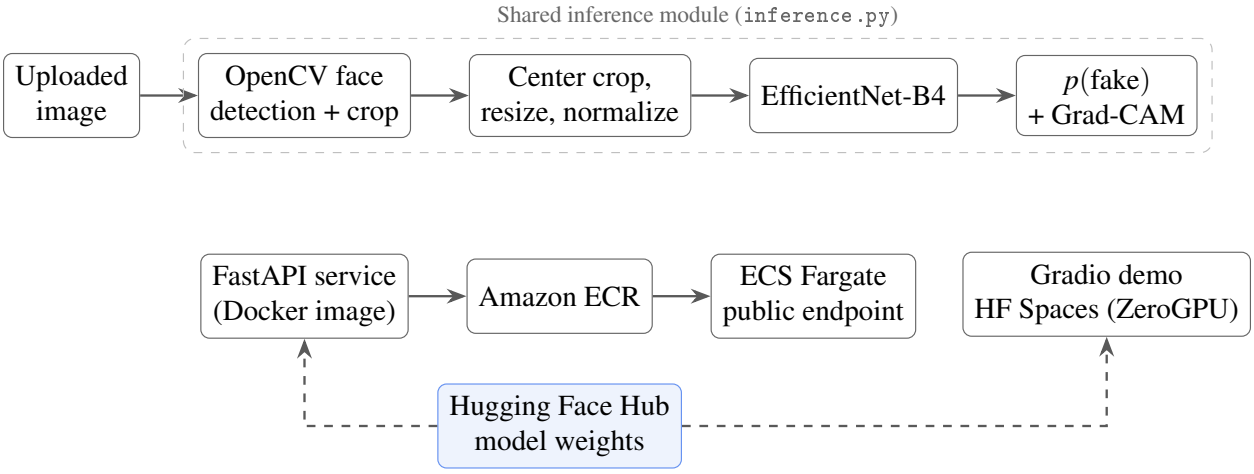


Figure 2: Inference pipeline (top) and serving architecture (bottom). Both serving paths use the same inference module and pull weights from the Hugging Face Hub.

6.1 Inference Pipeline

Training data consists of tight face crops, but real uploads are arbitrary photos. The inference module bridges that gap so the model sees inputs that match its training distribution:

1. **Face detection.** OpenCV's frontal-face Haar cascade [5] locates faces. The largest face is selected and cropped as a square with a 25% margin. If no face is found, the module falls back to a centered square crop and reports that in the response.
2. **Preprocessing.** The crop is resized to 260×260 to match the dataset, then passed through the exact evaluation transform from training (center crop to 208, resize to 224, ImageNet normalization).
3. **Prediction and explanation.** The model returns $p(\text{fake})$ and a label at a 0.5 threshold. Grad-CAM optionally produces an overlay on the preprocessed face.

The same module is used by both the REST API and the Gradio demo, so the two cannot drift apart.

6.2 Model Hosting

The trained weights (`deepsentinel_effb4.pt`) are published to the Hugging Face Hub at `adichaudhary/deepsentinel`. Both serving paths download weights from this single source, so updating the model requires only a new upload and a redeploy.

6.3 REST API

The API is built with FastAPI and exposes three endpoints:

Endpoint	Description
`GET /health`	Liveness check and model load status
`POST /predict`	Multipart image upload. Returns `label`, `p_fake`, face box, and an optional base64 Grad-CAM PNG (`?heatmap=true`)
`GET /docs`	Interactive OpenAPI documentation

The model loads once at startup through FastAPI's lifespan handler, so requests do not pay model-loading cost. Uploads are capped at 10 MB, and invalid files return HTTP 400. Responses are validated with Pydantic models. The following request was sent to the live ECS deployment with a held-out manipulated face from the test set:

```
curl -F "file=@sample_fake.png" "http://<ecs-endpoint>:8000/predict"

{"label": "fake", "p_fake": 0.9992, "face_detected": true,
 "face_box": [0, 0, 331, 331], "heatmap_png_base64": null}
```

Passing `?heatmap=true` adds the Grad-CAM overlay as a base64-encoded PNG.

6.4 Containerization

The service is packaged with Docker on a `python:3.12-slim` base. Design choices for the image:

- **CPU-only PyTorch** wheels keep the image far smaller than a CUDA build, and EfficientNet-B4 inference on a single image is fast enough on CPU for this workload.

- **Weights are baked in at build time** from the Hugging Face Hub, so containers start without a network fetch and every image is tied to a fixed model version.
- **Hardening:** the container runs as a non-root user, a `.dockerignore` allowlist keeps everything except the application code out of the build context, and a Docker `HEALTHCHECK` polls `/health`.

6.5 AWS ECS Deployment

The container runs on AWS ECS with the Fargate launch type, which removes the need to manage servers. A single script, `deploy/deploy.sh`, handles the full deployment and is safe to rerun:

1. Creates an Amazon ECR repository if needed, then builds the image for `linux/amd64` and pushes it.
2. Creates the ECS task execution role with the `AmazonECSTaskExecutionRolePolicy` if it does not exist.
3. Registers a task definition (1 vCPU, 3 GB memory) with a container health check and CloudWatch Logs through the `awslogs` driver.
4. Creates the ECS cluster and a security group that allows inbound traffic on port 8000 in the default VPC.
5. Creates the Fargate service with a public IP, or forces a new deployment if the service already exists.
6. Waits for the service to stabilize and prints the public endpoint.

A companion script, `deploy/teardown.sh`, scales the service to zero and removes the service and cluster to stop billing while keeping the image in ECR for fast redeploys.

6.6 Interactive Demo

A Gradio application on Hugging Face Spaces gives anyone a browser-based way to try the model. Users upload a photo and receive class probabilities and a Grad-CAM overlay. The Space runs on ZeroGPU hardware, which attaches a GPU only while a request is being processed. Figure 3 shows the deployed demo on two held-out test images.

7 Limitations and Future Work

- **Single dataset.** Results come from one dataset. Cross-dataset evaluation on benchmarks such as FaceForensics++ [6] and Celeb-DF [7] would test generalization to unseen manipulation methods.
- **Still images only.** The model scores individual frames. Video detection could aggregate frame scores or use temporal models to catch flicker and inconsistencies across frames.
- **Training subset.** Only 20K of the roughly 100K available training images were used. Training on the full split and at the native B4 resolution of 380×380 is likely to improve results.
- **Face detector.** The Haar cascade is fast but misses profile and heavily occluded faces. A learned detector would improve robustness in production.
- **Calibration.** The 0.5 threshold is not tuned. Calibrating probabilities and choosing a threshold for a target false positive rate would suit real moderation workflows better.

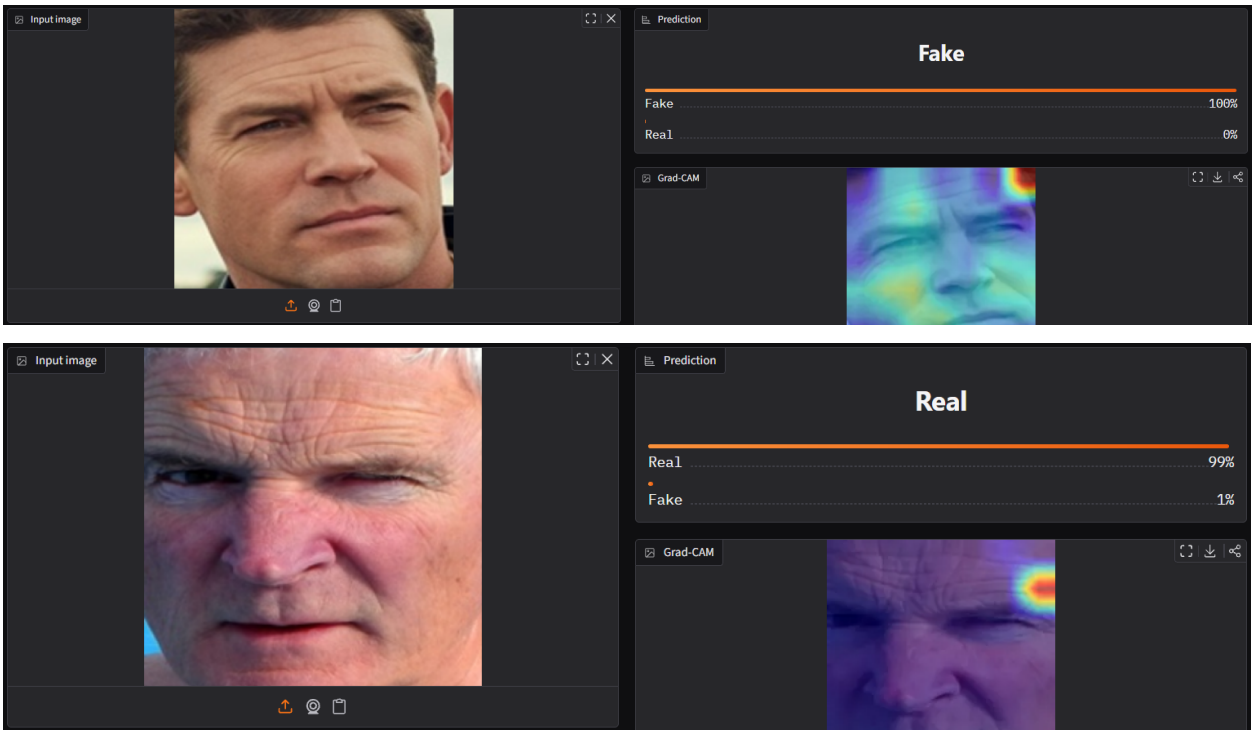


Figure 3: The live Hugging Face Spaces demo on held-out test images. Top: a manipulated face classified as fake with 100% confidence. Bottom: a real face classified as real with 99% confidence. Each result includes a Grad-CAM overlay.

8 Conclusion

DeepSentinel shows a complete path from research to deployment for deepfake detection. A fine-tuned EfficientNet-B4 reaches 0.956 ROC AUC on identities it has never seen. Grad-CAM analysis, paired with border removal and a masking ablation, confirms that the model does not rely on a spurious corner artifact. The model is served through a containerized FastAPI service on AWS ECS Fargate and a public Gradio demo, both sharing one inference module and one source of model weights.

References

[1] ThinothW. *Deepfake Identity Isolated Dataset (PreP)*. Hugging Face. `https://huggingface.co/datasets/ThinothW/Deepfake-Identity-Isolated-Dataset-PreP`

[2] M. Tan and Q. V. Le. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. In *ICML*, 2019.

[3] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra. Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization. In *ICCV*, 2017.

[4] R. Wightman. PyTorch Image Models. `https://github.com/huggingface/pytorch-image-models`, 2019.

[5] P. Viola and M. Jones. Rapid Object Detection using a Boosted Cascade of Simple Features. In *CVPR*, 2001.

[6] A. Rössler, D. Cozzolino, L. Verdoliva, C. Riess, J. Thies, and M. Nießner. FaceForensics++: Learning to Detect Manipulated Facial Images. In *ICCV*, 2019.

[7] Y. Li, X. Yang, P. Sun, H. Qi, and S. Lyu. Celeb-DF: A Large-scale Challenging Dataset for DeepFake Forensics. In *CVPR*, 2020.